클로드 코드 채용 심사단 프롬프트 실습

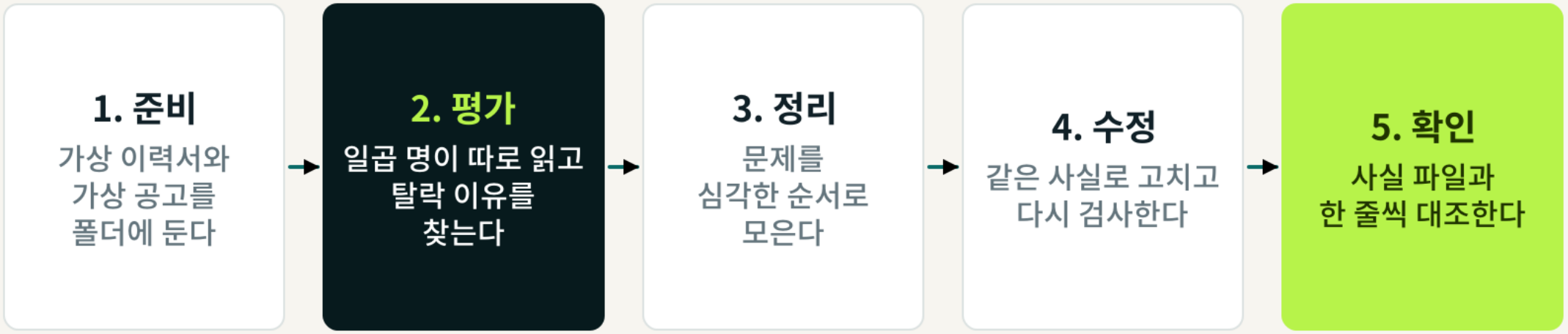
일곱 명의 AI 평가자로 이력서를 고친다



누구-어디-입력-출력-피드백 실습

가상 이력서를 평가받아 고친 뒤 나만의 심사단까지 만듭니다

PART 1에서 준비하고, PART 2에서 평가와 수정을 하고, PART 3에서 스킬을 직접 만든다



1~4는 클로드 코드가 하고, 5는 내가 클로드 코드에게 검사를 시켜 확인한다

입력은 모두 클로드 코드에게 시키는 문장이고, 나는 붙여넣고 확인만 합니다

모든 실습 장은 누구, 어디, 입력, 출력, 피드백 다섯 칸으로 읽습니다

입력 칸의 문장을 그대로 붙여넣고, 출력이 다르게 나오면 피드백 칸의 문장을 넣는다



누구
누가 누구에게 시키나

어디
어느 창에 입력하나

입력
붙여넣을 문장

출력
성공했을 때 화면

피드백
막혔을 때 넣을 문장

입력 칸의 문장은 실습 키트의 prompts.md에도 똑같이 들어 있습니다

실습에 나오는 낯선 말은 네 가지뿐입니다

모두 프로그램 이름이나 작업 방식의 이름이다 — 코드를 몰라도 된다

터미널

글로 명령하는 창

마우스 대신 명령을 글로 입력한다

클로드 코드

내 폴더에서 일하는 AI

파일을 읽고 만들고 고친다

스킬

작업 설명서 한 장

SKILL.md 파일을 /이름 으로 부른다

하위 에이전트

따로 일하는 도우미

서로의 대화를 보지 못한다

채용 심사단은 스킬 한 장이 하위 에이전트 일곱 명을 부리는 구조입니다

평가자를 나누는 이유는 앞사람 의견에 끌려가지 않게 하려는 것입니다

앵커링(anchoring): 처음 들은 판단이 뒤의 판단을 끌어당기는 현상

한 명이 일곱 역할을 맡으면

의견이 닮아 간다

- 첫 의견이 기준이 된다
- 비슷한 지적만 나온다

놓치는 문제가 생긴다



일곱 명이 따로 읽으면

서로 다른 이유가 나온다

- 각자 처음부터 읽는다
- 모은 뒤에 합친다

문제를 넓게 찾는다

시험 감독이 학생들을 떨어뜨려 앉히는 것과 같은 원리입니다

심사단은 먼저 거르는 두 명과 따로 읽는 다섯 명으로 이루어집니다

일곱 명은 모두 하위 에이전트이며, 이력서와 공고만 보고 판정과 근거를 낸다



평가 한 번에 하위 에이전트를 여덟 번 부릅니다 — 일곱 명에 수정본 재검사 한 번

실습에 필요한 가상 자료는 키트 하나에 모두 들어 있습니다

인물, 회사, 학교, 연락처는 모두 실습용으로 지어낸 것이다

hiring-lab/	실습 폴더 (STEP 2에서 만든다)
└ data/	
└ resume_kim_haneul.md	가상 지원자 김하늘의 이력서 (일부러 약하게 썼다)
└ job_posting_bright_bakery.md	가상 회사의 SNS 마케터 채용 공고
└ facts_kim_haneul.md	김하늘이 실제로 한 일과 하지 않은 일
└ essay_kim_minseo.md	가상 학생 김민서의 자기소개서
└ facts_kim_minseo.md	김민서가 실제로 한 일
└ reference/essay-panel/SKILL.md	PART 3 모범 답안
└ prompts.md	이 덱의 프롬프트를 모두 모은 파일

사실 파일은 수정본이 지어낸 내용을 잡아내는 정답지 역할을 합니다

01

준비

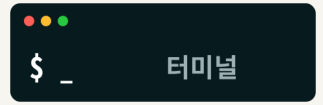
클로드 코드를 열고 실습 키트와 채용 심사단 스킬을 설치한다

STEP 1 · 클로드 코드가 설치되어 있는지 버전부터 확인합니다

버전 확인 명령은 클로드 코드를 켜지 않고 터미널에서 바로 입력한다

누구 나 → 컴퓨터에 직접 입력한다

어디 터미널 (윈도우는 PowerShell)



입력 — 터미널에 그대로 입력한다

```
claude --version
```

출력 — 이렇게 나오면 다음 단계로

숫자 뒤에 (Claude Code)가 붙은 한 줄이 나온다.

예: 2.1.302 (Claude Code)

피드백 — 다르게 나오면 이렇게 한다

command not found가 나오면 설치 명령을 입력한다.

맥: `curl -fsSL https://claude.ai/install.sh | bash`

윈도우: `irm https://claude.ai/install.ps1 | iex`

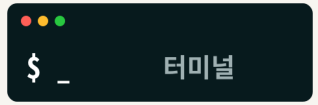
설치한 뒤에는 터미널 창을 새로 열고 다시 확인한다.

STEP 2 · 실습 폴더를 만들고 그 폴더에서 클로드 코드를 엽니다

클로드 코드는 연 폴더 안에서만 파일을 만들고 결과를 저장한다

누구 나 → 컴퓨터에 직접 입력한다

어디 터미널 (윈도우는 PowerShell)



입력 — 터미널에 그대로 입력한다

```
mkdir ~/hiring-lab && cd ~/hiring-lab && claude      ← 맥
mkdir ~/hiring-lab; cd ~/hiring-lab; claude          ← 윈도우
```

출력 — 이렇게 나오면 다음 단계로

클로드 코드 시작 화면이 열리고 아래에 입력창이 보인다.

처음이면 브라우저가 열리며 로그인을 요청한다.

피드백 — 다르게 나오면 이렇게 한다

이 폴더를 믿느냐고 물으면 hiring-lab 폴더인지 확인하고 Yes를 고른다.

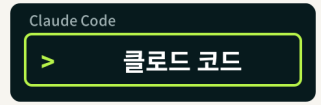
already exists가 나오면 폴더가 이미 있다는 뜻이니 cd ~/hiring-lab 와 claude 만 입력한다.

STEP 3 · 실습 키트 압축 풀기는 클로드 코드에게 맡깁니다

실습 키트에는 가상 이력서, 가상 공고, 사실 확인 파일, 모범 답안이 들어 있다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
다운로드 폴더에 있는 Hiring_Panel_Lab_Kit.zip 파일을 지금 이 폴더에 풀어 줘.
다 풀었으면 폴더 안의 파일 목록을 트리 모양으로 보여 줘.
```

출력 — 이렇게 나오면 다음 단계로

data, reference 두 폴더와 prompts.md 파일이 트리로 보인다.

data 폴더에는 md 파일 다섯 개가 들어 있다.

피드백 — 다르게 나오면 이렇게 한다

zip 파일을 찾지 못했다고 하면 이렇게 입력한다.

```
"내 컴퓨터에서 Hiring_Panel_Lab_Kit.zip 파일이 어디 있는지 찾아서 그 파일을 이 폴더에 풀어 줘."
```

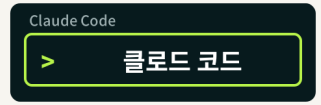
작업을 허락할지 물으면 내용을 읽고 Yes를 고른다.

STEP 4 · 채용 심사단 스킬 설치도 문장 한 번으로 끝냅니다

스킬은 ~/.claude/skills 아래 자기 이름의 폴더에 있어야 명령으로 불린다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
https://github.com/alexyc9381/hiring-panel-skill 저장소에 있는 hiring-panel 스킬을
~/.claude/skills/hiring-panel 폴더에 설치해 줘.
설치가 끝나면 SKILL.md 파일이 제자리에 있는지 확인하고 결과를 세 줄로 알려 줘.
```

출력 — 이렇게 나오면 다음 단계로

설치한 위치와 SKILL.md 확인 결과가 세 줄로 나온다.

마지막 줄에 /hiring-panel 명령을 쓸 수 있다는 안내가 나온다.

피드백 — 다르게 나오면 이렇게 한다

같은 이름의 스킬이 이미 있다고 하면 이렇게 입력한다.

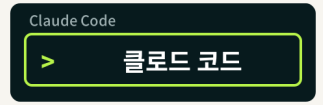
"이미 있는 스킬은 지우지 말고, 지금 들어 있는 SKILL.md 가 원본 저장소와 같은지만 확인해 줘."

STEP 5 · 클로드 코드를 다시 열어 /hiring-panel 명령이 보이는지 확인합니다

스킬 목록은 클로드 코드가 시작할 때 읽힌다 — 그래서 한 번 껐다가 다시 켠다

누구 나 → 컴퓨터에 직접 입력한다

어디 입력창에서 끄고 다시 켠다



입력 — 터미널에 그대로 입력한다

```
/exit              ← 클로드 코드 입력창에서 종료
claude             ← 같은 터미널에서 다시 실행
/hiring-panel      ← Enter는 누르지 말고 목록만 확인
```

출력 — 이렇게 나오면 다음 단계로

입력창 아래 명령 후보 목록에 /hiring-panel 이 보인다.

피드백 — 다르게 나오면 이렇게 한다

목록에 보이지 않으면 Esc로 입력을 지우고 이렇게 입력한다.

"hiring-panel 스킬이 명령 목록에 보이지 않아. 설치 위치와 SKILL.md 첫 부분의 name 줄을 점검하고 고쳐 줘."

02

평가와 수정

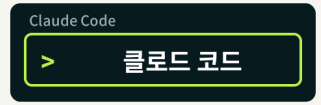
가상 이력서를 평가받고, 사실 파일로 검증하고, 다시 평가받는다

STEP 6 · 심사단을 부르기 전에 가상 이력서의 약점을 먼저 예상해 둡니다

가상 지원자 김하늘이 가상 회사 브라이트 베이커리의 SNS 마케터 자리에 지원한다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
data 폴더의 resume_kim_haneul.md 와 job_posting_bright_bakery.md 를 읽어 줘.
내가 채용 담당자라면 이 이력서에서 무엇이 가장 아쉬울지 세 문장으로 말해 줘.
```

출력 — 이렇게 나오면 다음 단계로

아쉬운 점 세 문장이 나온다.

예: 성과를 보여 주는 숫자가 하나도 없다.

피드백 — 다르게 나오면 이렇게 한다

세 문장을 메모장에 붙여 둔다.

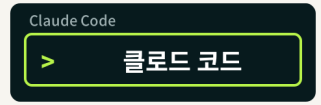
STEP 8에서 심사단 일곱 명이 찾은 문제와 비교하면 혼자 읽을 때 놓친 점이 보인다.

STEP 7 · 가상 이력서와 가상 공고를 넣어 심사단을 실행합니다

공고를 함께 주면 공고가 요구하는 경험과 이력서를 직접 맞대어 본다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
/hiring-panel data/resume_kim_haneul.md for this job: data/job_posting_bright_bakery.md
```

출력 — 이렇게 나오면 다음 단계로

몇 분 뒤 평가자 일곱 명의 판정이 한 줄씩 나온다.

가장 심각한 문제 다섯 개, 수정 전후 자동 검사 판정, 결과 파일 위치가 이어서 나온다.

피드백 — 다르게 나오면 이렇게 한다

지원 직무를 물으면 "브라이트 베이커리 주니어 SNS 마케터"라고 답한다.

사용량 한도 안내가 나오면 안내된 시각까지 기다렸다가 같은 문장을 다시 입력한다.

결과 화면에는 평가자마다 판정 한 줄과 이유 한 줄이 나옵니다

아래는 예시 화면이다 — 실제 문구와 판정은 실행할 때마다 달라진다

실행 결과 예시 — 실제 문구와 판정은 실행할 때마다 달라진다

자동 검사	REJECT	공고의 핵심 경험이 숫자로 보이지 않는다
채용 관리자	NO	'SNS 운영' 네 글자로는 무슨 일을 했는지 모른다
팀장	MAYBE	릴스를 직접 만들 수 있는지 확인이 필요하다
경쟁 지원자	PASS	내 이력서에는 팔로워 증가 수치가 있다
CEO	PASS	매장 매출이나 방문객에 준 영향이 없다
채용 담당자	PASS	첫 문단이 누구에게나 쓸 수 있는 말이다
동료	MAYBE	누구와 어떻게 일했는지가 보이지 않는다

결과 파일: .hiring-panel/2026-10-31-1/PANEL.md RESUME-IMPROVED.md

다섯 평가자의 PASS는 통과가 아니라 '채용하지 않음'이라는 뜻입니다

판정 기호는 평가자 종류에 따라 뜻이 다릅니다

판정은 영어 그대로 표시된다 — 같은 PASS라도 누가 냈는지 먼저 본다

자동 검사	**PASS 또는 REJECT**	사람에게 넘길지, 여기서 떨어뜨릴지
채용 관리자	**INTERVIEW 또는 NO**	면접을 볼 기회를 줄지
나머지 다섯 명	**HIRE, MAYBE, PASS**	뽑는다, 고민된다, 뽑지 않는다
`[ADD: ...]`	**수정본의 빈칸**	내가 사실을 채워야 하는 자리
`assumed`	**추정 표시**	평가자가 짐작한 부분이니 사실인지 확인

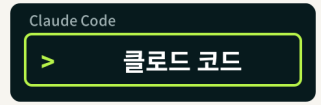
PASS를 보면 먼저 그 판정을 낸 평가자가 누구인지 확인합니다

STEP 8 · 결과 파일은 직접 뒤지지 않고 표로 정리하게 합니다

PANEL.md에는 일곱 명의 판정과 심각한 순서로 정렬한 문제 목록이 들어 있다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
방금 만들어진 .hiring-panel 폴더의 PANEL.md를 열어서
가장 심각한 문제 세 개를 표로 정리해 줘.
표의 열은 '문제가 된 문장', '탈락 이유', '지적한 평가자'로 해 줘.
```

출력 — 이렇게 나오면 다음 단계로

세 줄짜리 표가 나온다.

'SNS 운영'처럼 짧게 쓴 경력 줄이 위쪽에 오를 가능성이 크다.

피드백 — 다르게 나오면 이렇게 한다

STEP 6에서 적어 둔 세 문장과 비교한다.

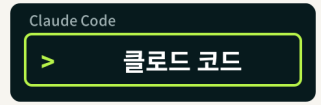
혼자서는 못 찾은 문제가 무엇이었는지 한 줄로 메모해 둔다.

STEP 9 · 수정본에 새로 들어간 문장부터 골라내게 합니다

수정본은 문장을 매끄럽게 고치면서 원본에 없던 숫자나 도구를 넣을 수 있다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
RESUME-IMPROVED.md 를 원본 data/resume_kim_haneul.md 와 한 문장씩 비교해 줘.
원본에 없던 숫자, 도구, 업무가 새로 들어간 문장을 모두 찾아서 표로 보여 줘.
```

출력 — 이렇게 나오면 다음 단계로

새로 들어간 문장 목록이 표로 나온다.

[ADD: ...] 로 비워 둔 칸도 함께 표시된다.

피드백 — 다르게 나오면 이렇게 한다

표에 나온 문장이 사실인지는 다음 단계에서 사실 파일로 확인한다.

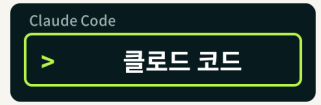
표가 비어 있으면 "[ADD 칸과 assumed 표시가 붙은 문장만 따로 보여 줘"라고 입력한다.

STEP 10 · 사실 파일에 적힌 내용만으로 빈칸을 채웁니다

facts_kim_haneul.md는 김하늘이 실제로 한 일과 하지 않은 일을 적은 가상 사실 파일이다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
data/facts_kim_haneul.md 에 적힌 사실만 사용해서 RESUME-IMPROVED.md 를 고쳐 줘.
사실 파일에 없는 숫자나 도구는 지우고, 채울 사실이 없는 [ADD] 칸은 그대로 남겨 줘.
결과는 이 폴더 맨 위에 RESUME-FINAL.md 로 저장하고 바뀐 문장 목록을 보여 줘.
```

출력 — 이렇게 나오면 다음 단계로

RESUME-FINAL.md 파일이 생기고 바뀐 문장 목록이 나온다.

예: 팔로워 1,200명을 3,400명으로 늘렸다는 문장이 들어간다.

피드백 — 다르게 나오면 이렇게 한다

마지막으로 한 번 더 검사하게 한다.

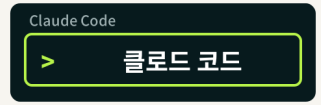
"RESUME-FINAL.md 에 사실 파일에 없는 내용이 하나라도 남아 있는지 다시 검사해 줘."

STEP 11 · 고친 이력서를 같은 심사단에게 다시 평가받습니다

같은 공고, 같은 평가자에게 다시 맡겨야 고친 효과를 공정하게 비교할 수 있다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
/hiring-panel RESUME-FINAL.md for this job: data/job_posting_bright_bakery.md
▶ 평가가 끝나면 이어서: 첫 번째와 두 번째 평가의 판정을 평가자별로 나란히 표로 비교해 줘.
```

출력 — 이렇게 나오면 다음 단계로

번호가 하나 늘어난 새 결과 폴더가 생긴다.

일곱 명의 전후 판정이 한 표에 나온다.

피드백 — 다르게 나오면 이렇게 한다

판정이 그대로인 평가자가 있으면 이렇게 입력한다.

"그 평가자의 지적을 사실 파일 안에서 고칠 방법을 알려 줘."

03

나만의 심사단 만들기

같은 구조로 자기소개서 심사단 스킬을 직접 만든다

스킬 파일은 이름표, 일하는 순서, 지킬 규칙 세 부분으로 이루어집니다

SKILL.md는 클로드가 읽는 작업 설명서이며, 코드가 아니라 문장으로 쓴다

```
---
name: essay-panel
description: 자기소개서를 …
---
# /essay-panel
1. 글과 지원 학과 확인
2. 다섯 평가자를 동시에
3. 문제 목록으로 모은다
4. 수정본을 쓴다
5. 결과를 저장한다
## 규칙
- 지어내지 않는다
```

이름표
명령 이름과 언제 쓰는지 적는다

일하는 순서
번호를 붙여 단계별로 적는다

지킬 규칙
하지 말아야 할 일을 적는다

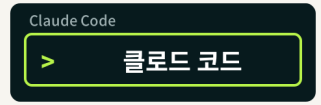
name 줄에 적은 이름이 그대로 /명령 이름이 됩니다

STEP 12 · 원본 스킬이 일하는 순서를 쉬운 말로 풀어 달라고 합니다

나만의 심사단을 만들기 전에 원본 SKILL.md의 뼈대를 먼저 이해한다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
~/.claude/skills/hiring-panel/SKILL.md 를 읽고, 이 스킬이 어떤 순서로 일하는지
고등학생이 이해할 수 있게 일곱 단계로 설명해 줘.
평가자들이 서로의 의견을 못 보게 만드는 문장이 어디 있는지도 그대로 인용해 줘.
```

출력 — 이렇게 나오면 다음 단계로

일곱 단계 설명이 나온다.

"Nobody sees anybody else's notes"
같은 원문 문장이 인용된다.

피드백 — 다르게 나오면 이렇게 한다

설명이 어려우면 이렇게 입력한다.

"방금 설명을 학교 시험 감독에 비유해서 다시 설명해 줘."

만들 심사단은 다섯 명이 자기소개서를 따로 읽는 구조입니다

PART 2의 채용 심사단에서 평가자의 역할과 판정 기호만 바꾼다



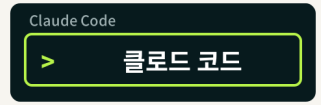
평가자 역할만 바꾸면 발표 대본이나 보고서 심사단도 같은 방법으로 만듭니다

STEP 13 · 같은 구조로 자기소개서 심사단을 설계만 먼저 받습니다

파일부터 만들면 고치기 어렵다 — 표로 설계를 보고 확인한 뒤에 만든다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
hiring-panel과 같은 구조로 자기소개서 심사단을 설계해 줘.
평가자는 입학사정관, 지원 학과 교수, 같은 학과에 지원한 다른 학생, 담임 선생님,
처음 읽는 일반 독자 다섯 명으로 하고, 각자 무엇을 보고 어떤 판정을 내리는지 표로 보여 줘.
아직 파일은 만들지 마.
```

출력 — 이렇게 나오면 다음 단계로

평가자마다 보는 점과 판정 기호가 표로 나온다.

파일은 아직 생기지 않는다.

피드백 — 다르게 나오면 이렇게 한다

고치고 싶은 칸은 바로 말한다.

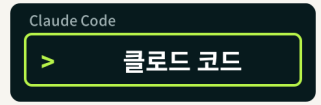
"담임 선생님은 실제로 한 일처럼 구체적인지 보게 바꿔 줘."

STEP 14 · 설계를 확정하고 SKILL.md 파일을 만들게 합니다

아래 세 규칙이 빠지면 심사단이 아니라 평범한 첨삭기가 된다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
방금 설계대로 ~/.claude/skills/essay-panel/SKILL.md 를 만들어 줘. 다음 규칙을 꼭 넣어 줘.
1) 다섯 평가자는 한 번에 동시에 부르고, 서로의 의견을 보지 못하게 한다.
2) 원문에 없는 경험이나 숫자는 지어내지 않고, 모르는 곳은 [추가: …] 로 남긴다.
3) 결과는 .essay-panel 폴더에 PANEL.md 와 ESSAY-IMPROVED.md 로 저장한다.
```

출력 — 이렇게 나오면 다음 단계로

SKILL.md 파일이 만들어졌다는 안내와 파일 내용 요약이 나온다.

피드백 — 다르게 나오면 이렇게 한다

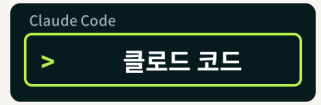
파일 쓰기를 허락할지 물으면 경로가 essay-panel 폴더인지 보고 Yes를 고른다.

STEP 15 · 클로드 코드를 다시 열고 내가 만든 /essay-panel을 실행합니다

가상 학생 김민서의 기계공학과 자기소개서로 시험해 본다

누구 나 → 클로드 코드에게 시킨다

어디 다시 연 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
/essay-panel data/essay_kim_minseo.md
지원 학과는 기계공학과야. 김민서가 실제로 한 일은 data/facts_kim_minseo.md 에 있어.
```

출력 — 이렇게 나오면 다음 단계로

평가자 다섯 명의 판정이 한 줄씩 나온다.

.essay-panel 폴더에 PANEL.md 와 ESSAY-IMPROVED.md 가 생긴다.

피드백 — 다르게 나오면 이렇게 한다

명령이 보이지 않으면 STEP 5의 점검 문장에서 hiring-panel을 essay-panel로 바꿔 입력한다.

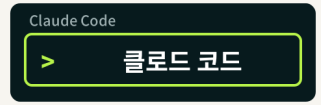
평가자가 다섯 명이 아니면 STEP 16에서 SKILL.md를 고친다.

STEP 16 · 결과에서 아쉬운 점을 말하면 클로드 코드가 SKILL.md를 고칩니다

스킬은 한 번에 완성되지 않는다 — 실행하고 고치기를 두세 번 반복한다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
평가 결과를 보니 다섯 평가자의 말이 너무 비슷해.
평가자마다 서로 다른 기준으로 읽도록 SKILL.md 의 평가자 설명을 고쳐 줘.
무엇을 바꿨는지 고치기 전과 후를 나란히 보여 줘.
```

출력 — 이렇게 나오면 다음 단계로

SKILL.md의 바뀐 부분이 전후 비교로 나온다.

피드백 — 다르게 나오면 이렇게 한다

고친 뒤에는 /exit 하고 claude 로 다시 열어 STEP 15를 한 번 더 실행한다.

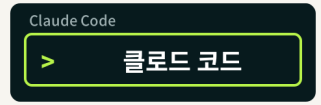
평가자들이 칭찬만 하면 "칭찬으로 시작하지 말라는 규칙을 넣어 줘"라고 입력한다.

STEP 17 · 모범 답안과 비교해 내 스킬에 빠진 규칙을 찾습니다

reference 폴더의 SKILL.md는 같은 과제를 미리 만들어 둔 모범 답안이다

누구 나 → 클로드 코드에게 시킨다

어디 클로드 코드 입력창



입력 — 클로드 코드 입력창에 그대로 붙여넣는다

```
reference/essay-panel/SKILL.md 와 내가 만든 ~/.claude/skills/essay-panel/SKILL.md 를 비교해서
내 파일에 빠진 규칙만 골라 알려 줘. 아직 고치지는 마.
```

출력 — 이렇게 나오면 다음 단계로

내 파일에 없는 규칙 목록이 나온다.

예: 지원 학과가 없으면 한 가지만 묻는다는 규칙

피드백 — 다르게 나오면 이렇게 한다

넣고 싶은 규칙만 골라서 이렇게 입력한다.

```
"방금 목록의 1번과 3번 규칙만 내 SKILL.md 에 넣어 줘."
```

막히면 증상을 그대로 알리는 문장 하나로 해결합니다

오류 메시지는 해석하지 않아도 된다 — 클로드 코드가 읽고 고친다

클로드 코드가 켜지지 않는다	터미널 창을 새로 열고 claude --version 부터 다시 입력한다
명령 목록에 스킬이 없다	"스킬이 명령 목록에 안 보여. 설치 위치를 점검하고 고쳐 줘."
파일을 못 찾는다고 한다	"이 폴더 안에서 그 파일이 어디 있는지 찾아서 경로를 알려 줘."
빨간 오류 글자가 나온다	"방금 나온 오류가 무슨 뜻인지 설명하고 고쳐 줘."
사용량 한도 안내가 나온다	안내된 시각까지 기다렸다가 같은 문장을 다시 입력한다

어두운 칸의 문장은 그대로 복사해 클로드 코드 입력창에 붙여넣습니다

스킬은 문장으로 쓴 작업 설명서다

평가자는 서로 못 보게 나눠야 한다

수정본의 사실은 사실 파일로 검사한다

역할만 바꾸면
어떤 글이든 심사단을 만든다

※ 심사단의 평가는 언어 모델이 만든 참고 의견이며 합격이나 채용을 보장하지 않는다. 실습 자료의 인물과 회사는 모두 가상이다.